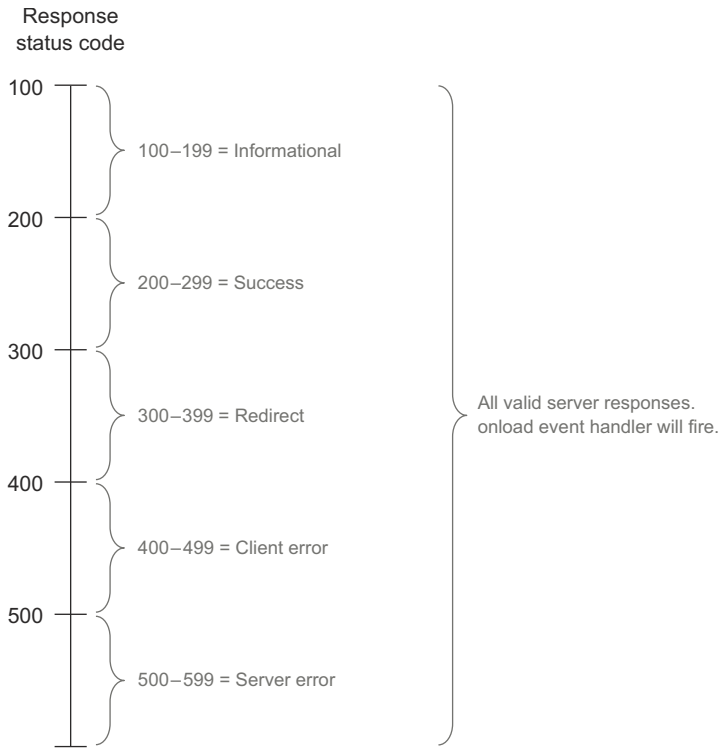


Figure 2.6 Distinction between the response and response status code. The status code may have an error, but the `onload` event handler will still fire.

Listing 2.5 shows how to add an `onerror` handler to the sample code. Note that while the `onerror` event signals that something went wrong with the cross-origin request, it doesn't tell you what went wrong. The `statusText` property will be empty, and the `status` will be 0. The code will only know that a request failed, and nothing more.

Listing 2.5 Handling a CORS error using the `onerror` event handler

```
<!DOCTYPE html>
<html>
<body onload="loadPhotos();">
<div id="photos">Loading photos...</div>
<script>
function loadPhotos() {
  var api_key = '<YOUR API KEY HERE>';
  var method = 'GET';
  var url = 'https://api.flickr.com/services/rest/?' +
      'method=flickr.people.getPublicPhotos&' +
      'user_id=32951986%40N05&' +
      'extras=url_q&format=json&nojsoncallback=1&' +
      'api_key=' + api_key;

  var xhr = new XMLHttpRequest();
  if (!('withCredentials' in xhr)) {
    alert('Browser does not support CORS.');
```